LAB 06 — FILE INTEGRITY MONITORING: AIDE

Baseline Establishment, Change Detection & Hash Verification

CySA+ Home Lab Portfolio | Alejandro W. Orellana | June 30, 2026

1. Lab Overview

This lab demonstrates file integrity monitoring (FIM) using AIDE (Advanced Intrusion Detection Environment) on Kali Linux. AIDE was built and configured from scratch, a cryptographic baseline was established across critical system directories, and two distinct change detection scenarios were tested: file addition and file modification with full hash comparison output.

Field	Detail
Tool	AIDE 0.19.3
Platform	Kali Linux — local filesystem monitoring
Config Location	/etc/aide/aide.conf (built manually)
Monitored Paths	/etc, /bin, /sbin, /usr/bin, /usr/sbin, /boot
Hash Algorithms	SHA256, SHA512, RMD160, GOST
Baseline Size	7,876 entries
ATT&CK; Techniques	T1105, T1565.001

2. AIDE Configuration

AIDE was not pre-installed with a default configuration on Kali — the config directory and file were built manually, defining monitored paths, hash algorithms, and excluded volatile directories (logs, temp files, process filesystems).

```
# /etc/aide/aide.conf
database_in=file:/var/lib/aide/aide.db
database_out=file:/var/lib/aide/aide.db.new
log_level=warning

NORMAL = sha256+rmd160+sha512+tiger+haval+gost+crc32+whirlpool

/etc NORMAL
/bin NORMAL
/sbin NORMAL
/usr/bin NORMAL
/usr/sbin NORMAL
/boot NORMAL

!/var/log
!/var/run
!/proc
!/sys
!/dev
!/tmp
```

> **Note:** This AIDE build (0.19.3) does not support tiger, crc32, haval, or whirlpool hash algorithms — these were automatically dropped with a warning, leaving SHA256, SHA512, RMD160, and GOST active. This is expected behavior on modern builds and does not weaken detection — four strong hash algorithms still provide robust collision resistance.

3. Baseline Database Initialization

```
$ sudo aide --init --config=/etc/aide/aide.conf
Start timestamp: 2026-06-26 10:58:40 -0700 (AIDE 0.19.3)
AIDE successfully initialized database.
New AIDE database written to /var/lib/aide/aide.db.new
Number of entries: 7876
End timestamp: 2026-06-26 10:59:25 -0700 (run time: 0m 45s)

$ sudo mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

Metric	Value
Baseline entries	7,876
Scan duration	45 seconds
Hash algorithms applied	SHA256, SHA512, RMD160, GOST
Database location	`/var/lib/aide/aide.db`

4. Detection Scenario 1 — File Addition

A test file was created to simulate an attacker dropping a new file on the system (e.g. a webshell, dropper, or staged payload).

```
$ sudo touch /etc/aide-test-file.txt
$ sudo aide --check --config=/etc/aide/aide.conf

AIDE found differences between database and filesystem!!
Summary:
Total number of entries: 7877
Added entries: 1
Removed entries: 0
Changed entries: 0

Added entries:
f++++++++++++++++++: /etc/aide-test-file.txt
```

> **Analysis:** The 'f' prefix identifies the entry type as a regular file. The string of '+' characters indicates every monitored attribute (size, permissions, hashes, timestamps) is new — exactly what AIDE reports when a file did not exist in the baseline. This maps to **T1105 (Ingress Tool Transfer)** — detecting newly introduced files is a primary FIM use case for catching malware staging.

5. Detection Scenario 2 — File Modification with Hash Comparison

An existing critical file (/etc/hosts) was modified to simulate an attacker altering system configuration — a common technique for local DNS redirection or persistence.

```
$ echo "# test modification" | sudo tee -a /etc/hosts
$ sudo aide --check --config=/etc/aide/aide.conf

AIDE found differences between database and filesystem!!
Summary:
Total number of entries: 7877
Added entries: 1
Removed entries: 0
Changed entries: 1

Changed entries:
f H : /etc/hosts

Detailed information about changes:
File: /etc/hosts
SHA256 : b9nHvTh6+0RgSga/LleR36nrBYZT3aof | XTSY1HAV7EWMsc8PFXnvadtgAqX1YHzp
mszS60o2UFU= | 4IVM57ZuGKU=
SHA512 : nYSf6OiiQzVmniWFxaT+xCtBlJcavZ7h | e9WaMog5oizgTNnoGVeWG2kC14sNKsQB
xNUcQ+4S4v4V3HshGWXhgq2yqewjqiOz | bcj+LdMP639GyKBi+IsAsy0jIcTt5LVL
RMD160 : pa/AhlERYuFFIZelvtsD+AVaC58= | OXrhj+24d4llFpPmrZKirSI8u7k=
GOST : 9nXZqbBTdGcMOrxZPHorDvVE10dlKfT1 | VRjfCuLGjKuh4eVRd37GkAom7djG1NLF
```

Analysis: The 'H' flag indicates a hash mismatch. AIDE displays before/after values side by side for every configured hash algorithm — SHA256, SHA512, RMD160, and GOST all show divergent values, cryptographically proving the file content changed. A modified /etc/hosts is a serious finding in production: attackers commonly alter this file for local DNS redirection to malicious IPs or to block security update domains. Maps to **T1565.001 (Stored Data Manipulation)**.

6. MITRE ATT&CK; Mapping

Technique ID	Name	Tactic	AIDE Detection
T1105	**Ingress Tool Transfer**	Command and Control	File addition detection (/etc/aide-test-file.txt) — catches newly staged tools, payloads, or webshells dropped on the filesystem
T1565.001	**Stored Data Manipulation**	Impact	File modification detection (/etc/hosts) — catches unauthorized changes to system configuration via cryptographic hash mismatch

7. Analyst Findings

FIM provides cryptographic proof, not just file metadata: Unlike simple timestamp or size monitoring, AIDE's hash-based comparison provides cryptographic certainty that file content changed. An attacker who modifies a file and resets its timestamp cannot evade hash-based FIM.

Multiple hash algorithms protect against algorithm-specific weaknesses: Running SHA256, SHA512, RMD160, and GOST simultaneously means a hypothetical collision attack against one algorithm would still be caught by the others — defense in depth at the cryptographic layer.

Baseline accuracy depends on initialization timing: AIDE's baseline is only as trustworthy as the system state at --init time. In production, baselines should be established immediately after a known-clean install, before the system is exposed to untrusted networks — otherwise a pre-existing compromise gets baked into the 'normal' state.

FIM is a detective control, not preventive: AIDE does not block file changes — it detects them after the fact during a scheduled --check run. In production this requires cron scheduling and alert integration (e.g. piping output to a SIEM) to provide timely detection rather than after-the-fact forensics.